Interaction System

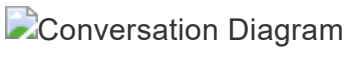
As a general purpose robot, Misty is designed to do many things. She's expressive, independently mobile, and very interactive. She's also highly programmable, which gives our community the ability to implement use cases we should never have thought of. The breadth of the platform sometimes comes at the cost of depth or ease of use, though those things tend to resolve as we get feedback from the community.

Why Does the Interaction System Exist?

As the community began to develop with Misty, a few trends clearly emerged, the clearest of which was that everyone wanted to orchestrate conversations. All of the basic building blocks have existed within Misty for some time, though creating those interactions was more difficult than it could be. Internally, we had created several different conversation management systems to solve these use cases, but never built anything into the robot. After working with the community, a clear need emerged to have a simple way to create dialogs and transition between them.

Architecture

The overall architecture consists of many moving pieces, but is different from the rest of the system in that it is stateful. Very few of Misty's commands produce a durable result, with the exception of file operations and a few configuration operations. With this system, the API produces artifacts that are persisted and reused between skills or interactions. Fundamentally, this system is a state-based workflow, meaning that there are discrete, defined states. The transitions between states are configuration data, and can be applied based on the needs of the interaction. Consider this diagram of a conversation.



As you can see from the diagram, states are declared and contain discrete actions to be taken. In addition, states transitions are mapped together through various types of triggers.

Contexts

Contexts within Misty are a construct designed to support natural language processing (NLP), and effectively represent a group of Intents. Internally, Misty uses a subset of the Apache OpenNLP framework as an embedded intent classifier. Misty comes preloaded with a default yes/no Context named "furhat.context.en.yes-no" that you can use, and of course you can create your own. To get started, you'll need to become familiar with a few key concepts.

Intents

An Intent represents the underlying meaning of a specific piece of text. This meaning is specified by the creator of the intent. When using conversational systems, you'll find that users can say many different things, but they ultimately all mean the same thing. You could certainly listen explicitly for "yes", "yeah", "affirmative", "of course", and anything else as a means of understanding that the user is just trying to say yes, but doing so is cumbersome to maintain and error prone.

Entities

In NLP systems, it's common to need specific pieces of text from a larger statement. As an example, consider adding something to a shopping cart. The user could say "I'd like 5 pineapples". It would be useful to know what kind of item the user would like, and also how many. In this example, both "5" and "pineapples" could represent entities. As a person creating an interaction with Misty, it would be ideal to receive those elements as variables, rather than be forced to parse the text yourself.

Creating Contexts

Now that you know the basics, let's look at how it works for Misty. To create a Context, use the TrainNLPEngine function. The API endpoint can be found at /api/dialogs/train. The message has a few required elements.

- Context - This is the name of the Context
- Save - This boolean determines if the Context should be persisted to storage for reuse, or if it should only exist in memory
- Overwrite - This boolean specifies that an existing Context with this name should be overwritten, if it exists
- Intents - This is an array of complex objects containing the Intent data

Intents have a minimum or two attributes:

- Name - This is the name of the intent. When handling intents, this is the value you'd filter on.
- Samples - This is an array of strings that represent the kinds of text that should be treated as this intent

In addition, an Intent may have one or more Entities. In those cases, the the Intent object would also contain an array or Entities.

Adding this all up, to create a Context through Misty's API, you'd send a message like this:

```
POST http://{ROBOT-IP-ADDRESS}/api/dialogs/train

{
    "Context": "my context",
    "Intents": [
        {
            "Name": "time",
            "Entities": [],
            "Samples": [
                "what time is it",
                "what is the time",
                "what time is it now"
            ]
        },
        {
            "Name": "date",
            "Entities": [],
            "Samples": [
                "what is the date",
                "what is today",
                "what day is it"
            ]
        }
    ]
}
```

JavaScript

Similarly, to create an Intent with Entities, you'd send a message like this:

```
POST http://{ROBOT-IP-ADDRESS}/api/dialogs/train

{
```

```
        "Context": "other context",
        "Overwrite": true,
        "Intents": [
            {
                "Name": "order",
                "Entities": [
                    {
                        "name": "quantity"
                    },
                    {
                        "name": "fruit"
                    }
                ],
                "Samples": [
                    "I'd like to order <START:quantity> 3 <END> <START:fruit>
pineapples <END>",
                    "I want <START:quantity> 10 <END> <START:fruit> oranges
<END>",
                    "I'll have <START:quantity> 1 <END> <START:fruit>
watermelon <END>"
                ]
            },
            {
                "Name": "favorite-color",
                "Entities": [
                    {
                        "name": "color"
                    }
                ],
                "Samples": [
                    "My favorite color is <START:color> blue <END>",
                    "I like the color <START:color> red <END>",
                    "<START:color> green <END> is my favorite color"
                ]
            }
        ]
}
```

JavaScript

> **Note:** The spaces inside of the entity START and END tags are significant. Currently, if the data cannot be parsed correctly, the call to train will contain a "result": false element.

Actions

While creating interactions for Misty, you'll quickly discover that leveraging the physicality of Misty makes for much better experiences. Additionally, you'll find that many of these animations are quite reusable and can create consistency in a user experience. To that end, Misty supports a construct called an "Action". Actions represent short, named, reusable animations that can be easily invoked. At the core of an action is a script that defines the desired behavior.

To get a list of Actions currently available, call the GetActions method:

```
GET http://{ROBOT-IP-ADDRESS}/api/actions
```

JavaScript

In the list that returns, the two most important elements are "name" and "script". The name is arbitrary, and represents the piece of data you'll use to invoke the Action later. The script attribute represents a simple script of commands to be sent executed on Misty. There are a few things to note about these scripts:

- Each command is uppercase
- Command parameters are specified after the command name, following a colon
- Command parameters are delimited by a comma
- Each command line is terminated with a semi-colon
- Actions prefixed with an octothorpe are treated as cleanup commands executed at the end of the animation (commonly things like RESET-EYES or RESET-LAYERS)

Here's a complete list of commands available to the script.

- ARMS:leftDegrees,rightDegrees,timeMs;
- ARMS-V:leftDegrees,rightDegrees,velocity;
- ARMS-OFFSET:leftDegrees,rightDegrees,timeMs; //offset commands are based off current actuator values
- ARMS-OFFSET-V:leftDegrees,rightDegrees,velocity;
- ARM:left/right,degrees,timeMs;
- ARM-V:left/right,degrees,velocity;
- ARM-OFFSET:left/right,degrees,timeMs;
- ARM-OFFSET-V:left/right,degrees,velocity;
- HEAD:pitch,roll,yaw,timeMs; //use null to not change a degree in head commands
- HEAD-OFFSET:pitch,roll,yaw,timeMs; //use 0 to not change a degree in head commands
- HEAD-V:pitch,roll,yaw,velocity; //use null to not change a degree in head commands
- HEAD-OFFSET-V:pitch,roll,yaw,velocity; //use 0 to not change a degree in head commands
- PAUSE:timeMs;
- VOLUME:newDefaultVolume;
- DEBUG: User websocket message to send if skill is debug level;
- PUBLISH: User websocket message to send;
- LIGHT:true/false/on/off;
- PICTURE:image-name-to-save-to,display-on-screen[,width,height]; optional width and height resize
- SERIAL:write to the serial stream;
- STOP;
- RESET-LAYERS; //clear user defined web, video, text and image layers
- RESET-EYES; //reset eyes and blinking to system defaults
- HALT;
- IMAGE:imageNameToDisplay.jpg; //displays on default eye layer
- IMAGE-URL:http://URL-to-display.jpg; //displays on default eye layer
- TEXT:text to display on the screen;
- CLEAR-TEXT;
- SPEAK:What to say; //can use generic data and inline speech, like 'Speak' in animations
- AUDIO:audio-file-name.wav;
- VIDEO:videoName.mp4;
- VIDEO-URL:http://videoName-to-play.mp4;
- CLEAR-VIDEO;
- WEB:http://site-name;
- CLEAR-WEB;
- LED:red,green,blue;
- LED-PATTERN:red1,green1,blue1,red2,green2,blue2,durationMs,blink/breathe/transit;
- START-LISTEN; //starts trying to capture speech
- SPEAK-AND-LISTEN;
- ALLOW-KEYPHRASE; //"Allows" keyphrase to work, but won't start if Misty is speaking or already listening and will wait until she can to allow keyphrase for the interaction
- CANCEL-KEYPHRASE; //turn off keyphrase rec
- SPEAK-AND-WAIT:What to say, timeoutMs;

- SPEAK-AND-EVENT:What to say,trigger,triggerFilter,text;
- SPEAK-AND-LISTEN:What to say; //starts listening after speaking the text
- FOLLOW-FACE;
- FOLLOW-OBJECT:objectName;
- STOP-FOLLOW;
- DRIVE:distanceMeters,timeMs,true/false(reverse);
- HEADING:heading,distanceMeters,timeMs,true/false(reverse);
- TURN:degrees,timeMs,right/left;
- ARC:heading,radius,timeMs,true/false(reverse);
- TURN-HEADING:heading,timeMs,right/left;
- RESPONSIVE-STATE:true/on/false/off; //if true, this interaction will respond to external bot events and commands, defaults to on
- HAZARDS-OFF;
- HAZARDS-ON;
- START-SKILL: skillId;
- STOP-SKILL: skillId;
- EVENT:trigger,triggerFilter,text;// send an event

To create an Action, send a message like this:

```
POST http://{ROBOT-IP-ADDRESS}/api/actions

{
       "editable": true,
       "name": "sigh",
       "script":
       "LED-PATTERN:0,0,255,40,0,112,1200,breathe;
       IMAGE:e_ApprehensionConcerned.jpg;
       ARMS:29,29,1000;
       HEAD:10,0,0,1000;"
}
```

JavaScript

To execute that action, send a message like this:

```
POST http://{ROBOT-IP-ADDRESS}/api/actions/start

{
       "name": "sigh"
}
```

JavaScript

States

Within the interaction system, a State is the central component driving the robot's output. Typically, States are used as a means of causing the robot to further drive an interaction, perhaps asking a question, or performing an Action. States are independent entities and can be reused as standalone interactivity components, or owned by a conversation. States are also designed to maximize flexibility, with the downside being that they can be created in ways that are either inert, or not useful. Parameters for a State are:

- Name - Required, unique name
- Overwrite - If a State with the provided name alreads exists, it is overwritten
- Speak - Something that the robot should say
- Audio - The file name of an audio clip that should be played
- Listen - A boolean indicating that the robot should listen for speech
- Contexts - One or more Contexts that should be used when listening for speech
- PreSpeech - An Action that should be performed while the speech recognizer is processing

- StartAction - An Action that should be perfomed at the beginning of the State
- SpeakingAction - An Action that should be performed while the robot is speaking
- ListeningAction - An Action that should be performed while the robot is listening
- TransitionAction - An Action to perform while the robot transitions to the next State
- NoMatchAction - An Action to perform if the NLP system doesn't successfully match an intent to the recognized speech
- NoMatchSpeech - What the robot should say if the NLP system doesn't successfully match an intent to the recognized speech
- NoMatchAudio - The file name of an audio clip that should be played if the NLP system doesn't successfully match an intent to the recognized speech
- RepeatMaxCount - The number of times that the robot should repeat the State if none of the transition triggers occur
- FailoverState - The State that the robot should transition to when RepeatMaxCount is exceeded
- Retrain - If the State consumes several Contexts, they can be retrained as a single Context for use by the State, potentially improving accuracy as the cost of latency in the beginning of the state
- ReEntrySpeech - If the State is re-triggered, this is used as a variation on the provided Speak text

> **Note:** In version 2.0.0 of the Misty platform, the Contexts attribute for a State has some undesirable behaviors. Specifically, the SDKs pass Context information as an array, but the REST API passes information as a string. This string is then interpreted as a JSON array, which results in an error. As such, the syntax for the Contexts element is requires that a JSON string array is passed as a string, i.e. "Contexts": "['some-context']"

To create a State, send a message like this:

```
POST http://{ROBOT-IP-ADDRESS}/api/states

{
     "Name": "com.furhatrobotics.demo.startstate",
     "Speak": "hello how are you?",
     "Contexts": "['how-are-you']",
     "Listen": true,
     "StartAction": "hi",
     "Overwrite": true
}
```

JavaScript

The State created would:

1. Start speaking the text "hello how are you?"
2. Play the Action named 'hi'
3. Listen for user speech, processing it through the 'how-are-you' Context

Mappings & Transitions

When designing interactions, it's necessary to determine when a state transition should occur. As such, Misty allows for the mapping of states to each other. These mappings contain several key pieces of data that help to determine when a transition should occur. The first key key element is called a StateTrigger. Each StateTrigger also supports limited parameterization through a TriggerFilter. The currently supported StateTriggers are:

- SpeechHeard - Occurs when a specific speech intent is detected
  - The TriggerFilter specifies the Intent received from the active Context
- BumperPerssed - Occurs when a bumper is depressed

    - The TriggerFilter specifies which bumper is depressed, or no filter for any bumper.
    - Bumpers are: Bumper_FrontRight, Bumper_FrontLeft, Bumper_BackRight, Bumper_BackLeft (brl, brr, bfl, bfr) TODO: Validate me
- BumperReleased - Occurs when a bumper is released
- CapTouched - Occurs when a capacitive touch sensor in Misty's head is triggered
    - The TriggerFilter specifies which capacitive touch sensor is triggered, or no filter for any of the sensors.
    - Capacitive touch sensors are: Cap_Back (headback), Cap_Chin (chin), Cap_Front (headfront), Cap_Left (headleft), Cap_Right (headright), Cap_Scruff (scruff)
- CapReleased - Occurs when a capacitive touch sensor in Misty's head is no longer triggered
- AudioCompleted - Occurs when Misty finishes speaking
    - The TiggerFilter specifies the utterance id for the text to speech command
- Timer - After a specific amount of time. This is useful for adding a timeout to a state
    - The TriggerFilter specifies the amount of time in milliseconds when the timer should elapse
- NewFaceSeen - Occurs when a new person is detected within the scene
    - The TiggerFilter specifies the label for the person, as specified during facial recognition training
- NewObjectSeen - Occurs when a new object is detected within the scene
    - The TriggerFilter specifies the object description, as returned by the object recognizer
- ExternalEvent - Occurs when an event is received requesting the transition
    - The TriggerFilter specifies the name of the event to listen for

When mapping states, the required arguments are:

- Conversation - The name of the Conversation
- Trigger - The type of StateTrigger
- State - The name of the state that is listening for the trigger
- NextState - The name of the state to transition to when the trigger occurs

Each state may have as many mappings as necessary to facilitate a given interaction.